

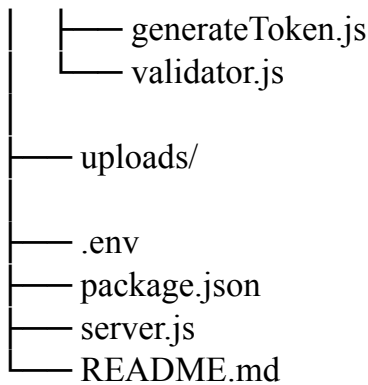
Backend Structure

Overview

The backend of the **DarshanEase** application is developed using **Node.js**, **Express.js**, and **MongoDB**, following the **MVC (Model–View–Controller)** architecture. This structure separates business logic, routing, database models, and middleware, making the application scalable, maintainable, and easy to understand.

Backend Folder Structure

```
backend/
├── config/
│   └── db.js
├── controllers/
│   ├── authController.js
│   ├── templeController.js
│   ├── bookingController.js
│   ├── slotController.js
│   └── adminController.js
├── middleware/
│   ├── authMiddleware.js
│   ├── adminMiddleware.js
│   └── errorMiddleware.js
├── models/
│   ├── User.js
│   ├── Temple.js
│   ├── Slot.js
│   ├── Booking.js
│   └── Admin.js
├── routes/
│   ├── authRoutes.js
│   ├── templeRoutes.js
│   ├── bookingRoutes.js
│   ├── slotRoutes.js
│   └── adminRoutes.js
└── utils/
```



Folder Description

1. config/

Contains configuration files required by the application.

Example

- db.js – Establishes the MongoDB database connection using Mongoose.

2. controllers/

Controllers contain the business logic. They process client requests, interact with models, and return responses.

Files

- authController.js – User registration and login.
- templeController.js – Temple management.
- bookingController.js – Booking operations.
- slotController.js – Darshan slot management.
- adminController.js – Administrative functions.

3. middleware/

Middleware executes before or after request processing.

Files

- authMiddleware.js – Verifies JWT tokens.
- adminMiddleware.js – Restricts admin-only routes.

- `errorMiddleware.js` – Handles application errors.

4. models/

Models define the MongoDB schemas and represent database collections.

Models

- User
- Temple
- Slot
- Booking
- Admin

Each model contains:

- Schema definition
- Validation rules
- Collection mapping

5. routes/

Routes define API endpoints and map them to controller functions.

Examples

- `/api/auth`
- `/api/temples`
- `/api/slots`
- `/api/bookings`
- `/api/admin`

6. utils/

Utility files contain reusable helper functions.

Examples

- JWT token generation
- Input validation
- Common helper functions

7. uploads/

Stores uploaded files if the application supports:

- Temple images
- User profile photos
- Documents

8. .env

Stores environment variables such as:

- PORT
- MONGODB_URI
- JWT_SECRET
- NODE_ENV

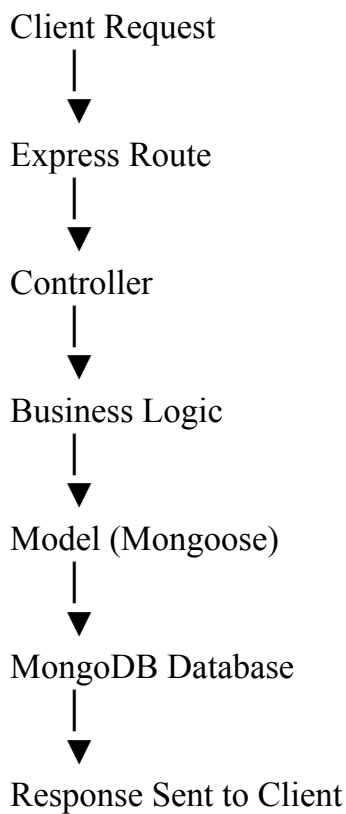
9. server.js

The main entry point of the application.

Responsibilities:

- Initialize Express
- Connect to MongoDB
- Register middleware
- Configure routes
- Start the server

Backend Workflow



MVC Mapping

Folder	MVC Component	Responsibility
models	Model	Database schemas and data operations
controllers	Controller	Business logic and request handling
routes	Controller Layer	Maps HTTP requests to controllers
frontend	View	User interface and presentation
middleware	Support Layer	Authentication, authorization, and error handling
config	Support Layer	Database and application configuration
utils	Support Layer	Reusable helper functions

Advantages of the Backend Structure

- Clear separation of concerns using MVC.
- Easy to maintain and debug.
- Supports scalable application development.
- Improves code reusability.
- Simplifies API management.

- Enhances security with middleware.
- Facilitates future integration of features such as online payments, notifications, and QR code verification.

Conclusion

The backend structure of DarshanEase follows industry-standard practices by organizing the application into dedicated folders for configuration, controllers, models, routes, middleware, and utilities. This modular architecture improves maintainability, supports team collaboration, and provides a strong foundation for building a secure and scalable temple darshan ticket booking system.